

Range-based **for** statements with initializer

Abstract

We propose a new versions of the range-based **for** statement for C++: `for (init; decl : expr)`. This statement simplifies common code patterns, help users keep scopes tight and offers an elegant solution to a common lifetime problem.

Contents

1. [Revision history](#)
2. [Before/After](#)
3. [Proposed wording](#)
4. [Discussion](#)
5. [Impact...](#)
 - 5.1 [... on the Standard](#)
 - 5.2 [... on implementations](#)
 - 5.3 [... on ongoing design work regarding multi-range iteration](#)
 - 5.4 [... on range adaptors and ongoing lifetime issues](#)
6. [Future directions](#)
7. [Acknowledgements](#)

Revision history

- [P0614r0](#): initial proposal
- P0614r1: simplified the formal wording to reuse the original range-based **for** rewrite rule

Before/After

Before the proposal	With the proposal
<pre> { T thing = f(); for (auto& x : thing.items()) { // Note: "for (auto& x : f().items())" is WRONG mutate(&x); log(x); } } </pre>	<pre> for (T thing = f(); auto& x : thing.items()) { mutate(&x); log(x); } </pre>
<pre> { std::size_t i = 0; for (const auto& x : foo()) { bar(x, i); ++i; } } </pre>	<pre> for (std::size_t i = 0; const auto& x : foo()) { bar(x, i); ++i; } </pre>

Proposed wording

Change the grammar in [stmt.iter] as follows.

iteration-statement:

```
...  
for ( init-statementopt for-range-declaration : for-range-initializer ) statement  
...
```

Modify paragraph 1 of subsection [stmt.ranged] as follows.

The range-based `for` statement

```
for ( init-statementopt for-range-declaration : for-range-initializer ) statement
```

is equivalent to

```
{  
    init-statementopt  
    auto &&__range = for-range-initializer ;  
    auto __begin = begin-expr ;  
    auto __end = end-expr ;  
    for ( ; __begin != __end; ++__begin ) {  
        for-range-declaration = *__begin;  
        statement  
    }  
}
```

where [...]

Discussion

This proposal shares a lot of the motivation of [P0305: Selection statements with initializer](#), namely the desire for tight scopes and local code. However, there is a more pressing motivation that is unique to the range-based `for` statement:

In a statement `for (auto& x : expr)`, the expression *expr* is evaluated once and bound to a notional variable declared as `auto&&`. When *expr* is a prvalue, this works well and the lifetime of the value is extended to beyond the loop. However, when *expr* is a glvalue, it will happily bind to the notional reference, but its lifetime is not extended and the reference is invalid. This pattern is a common source of bugs that is hard to spot; it is particularly easily caused by member functions that return glvalues (but also by reference-forwarding functions such as `std::min`).

For example, consider the following simple type that exposes an internal collection:

```
class T {  
    std::vector<int> data_;  
public:  
    std::vector<int>& items() { return data_; }  
    // ...  
};
```

Consider further a function returning a prvalue:

```
T foo();
```

Even users who are familiar with the intricacies of prvalue lifetime extension, and who would be confident about a hypothetical statement

```
for (auto& x : foo()) { /* ... */ }
```

can easily fail to spot that the similar looking

```
for (auto& x : foo().items()) { /* ... */ }
```

has undefined behaviour. While this particular pitfall will presumably stay with us for the foreseeable future (but [see below](#) for further discussion), the proposed new syntax will at least allow users to write correct code that looks almost as concise and local as the wrong code above:

```
for (T thing = foo(); auto& x : thing.items()) { /* ... */ }
```

Note that we are *not* proposing that the *init-statement* be in the same declarative region as any later part of the statement. In other words, `for (auto x = f(); auto x : x)` is valid and the outer `x` is hidden in the loop body. This is consistent with the proposed rewrite rule; in the current standard, `T x; for (auto x : x)` is already valid.

Impact...

... on the Standard

The proposal is a core language extension. The proposed syntax is ill-formed in the current standard. As an extension to the language’s statement syntax, this change is unlikely to have any impact on the design of the standard library.

... on implementations

Various implementers have reported that the proposal may pose certain implementation challenges, but should be doable in principle and in reasonable reality. The new syntax makes it harder to distinguish a range-based from an ordinary `for` statement and requires more sophisticated parsing to distinguish the two.

... on ongoing design work regarding multi-range iteration

Proposal [P0026](#) was presented in at the 2015 Kona meeting that proposes a syntax extension

```
for (auto x : a; auto y : b; auto z : c) {  
    f(a, b, c);  
}
```

which iterates the ranges `a`, `b` and `c` in lock-step (“zip”) order. Although that proposal has not progressed, raises several technical concerns (e.g. how to handle ranges of unequal length), and the problem it addresses can be solved in library, we would nonetheless like to note that the present proposal is compatible with and orthogonal to that extension: One could support an optional initializer syntax with multiple ranges just as well:

```
for (T thing = f(); auto x : a; auto y : b; auto z : c) {  
    thing.bar(a, b, c);  
}
```

That syntax is still syntactically unique, although it would prevent a future extension to allow an optional

increment statement. (The author has no intention of proposing such an extension.)

... on range adaptors and ongoing lifetime issues

It is pertinent to discuss a related set of range designs and current core and evolution issues. Let us revisit the motivating example where the lifetime of the range expression value ends prematurely because it is not a prvalue:

```
for (auto& x : f().things()) { // dangling reference!
    mutate(&x);
}
```

This problem is exacerbated if we consider a generic design of *range adaptors* which is a central component of many range-based libraries (and has been considered by the Ranges study group, SG9). Depending on the details, we may end up with *many* temporaries which all need to be kept alive:

```
for (auto& x : f().filter(pred).top(10).reverse()) { // how many temporaries?
    mutate(&x);
}
```

The proposed optional initializer is not sufficient to track all the temporary objects that may need to be kept alive during the iteration. In fact, this problem has been considered so serious that it is the subject of core issues [CWG 900](#) and [CWG 1498](#), and at the 2017 Kona meeting CWG decided to send these issues back to EWG for review. One of the possible solutions that has been considered is to give the range-based `for` statement special semantics by which *all* temporary values that are part of the range expression are alive until the end of the loop.

We would like to offer an alternative position and suggest that a core language change may not be needed here. First off, ongoing work in the Ranges study group has already come to the conclusion that range adaptors should not be constructible from rvalues. In such a design, the expression `f().filter(pred)` would not be allowed (assuming, as always, that `f()` is an rvalue). All we now need is that the entire state of the combined adaptor chain be accumulated in the final expression, and that that expression be a prvalue, so that no object except that of the final value in the adaptor chain is required during iteration. With that design constraint in place, and together with the present proposal, we can write iteration over an adapted range as follows:

```
for (T container = f(); auto& x : container.filter(pred).top(10).reverse()) {
    mutate(&x);
}
```

// lvalue prvalue prvalue prvalue

Future directions

Just for the record, without prejudice or promise, and with malice toward none, we would like to note possible future extensions in this area.

- Range-based `for` with increment: `for (init; auto x : e; inc)`
- Initializer for the `do` statement: The `do ... while` statement does not have an optional initializer at the moment. One might ask for one, though we would need to think hard whether `do (init;) statement while (expr);` is syntactically unambiguous and aesthetically defensible.

Acknowledgements

Thanks to Herb Sutter and Titus Winters for encouraging the proposal, to Eric Niebler and Ville Voutilainen for technical discussion regarding lifetime issues, and to all the implementers who provided feedback on the implementability of this idea.